

Java Interview Core Notes + Curated Questions with Solutions

Table of Contents

1. HashMap vs ConcurrentHashMap
 2. Interface vs Abstract Class
 3. REST Explained in Detail
 4. Thread vs Process
 5. ArrayList vs LinkedList
 6. HTTP Status Codes (Complete Reference)
 7. Curated Java Interview Questions with Solutions
-

1) HashMap vs ConcurrentHashMap

HashMap

- Not thread-safe.
- Allows one `null` key and multiple `null` values.
- Average time complexity for `put/get/remove` : $O(1)$.
- Worst-case can degrade when many collisions happen.

ConcurrentHashMap

- Thread-safe for concurrent read/write operations.
- Uses finer-grained synchronization and CAS style updates (not a global lock for every operation).
- Does not allow `null` keys or `null` values.
- Iterators are weakly consistent (no `ConcurrentModificationException` , and they may reflect partial concurrent updates).

Practical Decision Rule

- Use `HashMap` in single-threaded code.
- Use `ConcurrentHashMap` in shared mutable maps in multi-threaded apps.
- If a business operation requires multiple steps atomically, map thread safety alone is not enough; use higher-level synchronization/locking or redesign.

Common Interview Pitfalls

- Saying `HashMap` is "faster" always. It depends on concurrency and correctness requirements.
 - Assuming `ConcurrentHashMap` makes all multi-step workflows safe.
 - Using `null` with `ConcurrentHashMap` .
-

2) Interface vs Abstract Class

Interface

- Defines a contract/capability.
- Supports multiple inheritance of type.
- Can have `default` , `static` , and private methods in modern Java.
- No instance state (except constants).

Abstract Class

- Defines partial implementation + optional shared state.
- Single inheritance only.
- Can have constructors, fields, protected methods.
- Useful when subclasses share common lifecycle/stateful behavior.

How to Choose

- Pick interface when you want behavior contract across unrelated classes.
- Pick abstract class when there is real shared internal logic/state.
- Common enterprise pattern: interface for API + abstract base for reusable implementation.

Interview One-Liner

- Interface = what a type can do.
 - Abstract class = what a family of types is with shared internals.
-

3) REST Explained in Detail

What is REST

REST (Representational State Transfer) is an architectural style for building HTTP APIs around resources.

Core Principles

1. Resource-oriented URLs:
 - Good: `/users/42/orders`
 - Avoid verb-heavy endpoints like `/getUserOrders` .
2. Stateless requests:
 - Each request carries all context (auth, headers, payload).
3. Uniform interface:
 - Use HTTP methods and status codes semantically.
4. Representation:
 - Resources are represented as JSON (most common), XML, etc.
5. Cacheability:
 - Responses can be cached with headers (`Cache-Control` , `ETag`).
6. Layered architecture:
 - API gateways, load balancers, services, and DB layers can be composed cleanly.

HTTP Methods and Semantics

- `GET` : Read resource, safe and idempotent.
- `POST` : Create resource/action, not idempotent by default.
- `PUT` : Replace full resource, idempotent.
- `PATCH` : Partial update.
- `DELETE` : Delete resource, idempotent.

Idempotency and Safety

- Safe methods do not change server state by intention.
- Idempotent methods can be repeated without changing final effect.
- For `POST` , use idempotency keys in payment/order-style APIs.

Good REST Design Checklist

- Noun-based URI naming.
- Correct status code usage.
- Pagination (`limit` , `cursor` / `keyset`).
- Stable error schema (`code` , `message` , `traceId`).
- Versioning strategy (`/v1` or header-based).
- Security, observability, and rate limiting.

Example Endpoint Design

- `POST /orders` -> `201 Created`
 - `GET /orders/{id}` -> `200 OK` or `404 Not Found`
 - `PATCH /orders/{id}` -> `200 OK`
 - `DELETE /orders/{id}` -> `204 No Content`
-

4) Thread vs Process

Process

- Independent running program.
- Own memory space.
- Strong isolation from other processes.
- Communication via IPC (sockets, pipes, shared memory).
- Heavier context switching and startup than threads.

Thread

- Lightweight execution unit inside a process.
- Threads share process heap/resources.
- Each thread has its own stack and program counter.
- Faster to create/switch than processes.
- Needs synchronization to avoid race conditions.

Practical Trade-offs

- Need isolation/security boundary -> process.
 - Need fast concurrent tasks in same app -> threads.
 - Multi-threading introduces correctness risks: races, deadlocks, visibility bugs.
-

5) ArrayList vs LinkedList

ArrayList

- Dynamic array.
- Random access by index: $O(1)$.
- Append: amortized $O(1)$.
- Insert/delete in middle: $O(n)$ due to shifting.
- Better cache locality; usually faster in real systems.

LinkedList

- Doubly linked nodes.
- Random access by index: $O(n)$.

- Insert/delete near known iterator position: $O(1)$.
- Higher memory overhead per element.
- Poor cache locality, often slower in practice.

Rule of Thumb

- Default to `ArrayList` .
 - Use `LinkedList` only for very specific workloads with heavy iterator-local insert/remove and minimal indexing.
-

6) HTTP Status Codes (Complete Reference)

1xx Informational

- 100 Continue
- 101 Switching Protocols
- 102 Processing
- 103 Early Hints

2xx Success

- 200 OK
- 201 Created
- 202 Accepted
- 203 Non-Authoritative Information
- 204 No Content
- 205 Reset Content
- 206 Partial Content
- 207 Multi-Status
- 208 Already Reported
- 226 IM Used

3xx Redirection

- 300 Multiple Choices
- 301 Moved Permanently
- 302 Found
- 303 See Other
- 304 Not Modified
- 305 Use Proxy (deprecated)
- 306 (Unused)
- 307 Temporary Redirect
- 308 Permanent Redirect

4xx Client Error

- 400 Bad Request
- 401 Unauthorized
- 402 Payment Required
- 403 Forbidden
- 404 Not Found
- 405 Method Not Allowed
- 406 Not Acceptable
- 407 Proxy Authentication Required

- 408 Request Timeout
- 409 Conflict
- 410 Gone
- 411 Length Required
- 412 Precondition Failed
- 413 Content Too Large
- 414 URI Too Long
- 415 Unsupported Media Type
- 416 Range Not Satisfiable
- 417 Expectation Failed
- 418 I'm a teapot
- 421 Misdirected Request
- 422 Unprocessable Content
- 423 Locked
- 424 Failed Dependency
- 425 Too Early
- 426 Upgrade Required
- 428 Precondition Required
- 429 Too Many Requests
- 431 Request Header Fields Too Large
- 451 Unavailable For Legal Reasons

5xx Server Error

- 500 Internal Server Error
 - 501 Not Implemented
 - 502 Bad Gateway
 - 503 Service Unavailable
 - 504 Gateway Timeout
 - 505 HTTP Version Not Supported
 - 506 Variant Also Negotiates
 - 507 Insufficient Storage
 - 508 Loop Detected
 - 510 Not Extended
 - 511 Network Authentication Required
-

7) Curated Java Interview Questions with Solutions

A. HashMap / ConcurrentHashMap

Q1: Why can HashMap fail in concurrent writes?

Solution:

- HashMap does not synchronize structure mutations.
- Concurrent writes can create inconsistent bucket state.
- Use `ConcurrentHashMap` or external locking.

Q2: Why does ConcurrentHashMap disallow null keys/values?

Solution:

- In concurrent lookups, `null` is used to represent absence.

- Allowing `null` would make `get()` semantics ambiguous.

Q3: Is `computeIfAbsent` always called once?

Solution:

- Under race, mapping function may run more than once in some scenarios.
- Keep function side-effect free or idempotent.

Q4: HashMap complexity in Java 8+

Solution:

- Average $O(1)$, worst-case can degrade.
- Treeification of heavily-collided buckets helps avoid pure $O(n)$ chains.

Q5: Why mutable key is dangerous?

Solution:

- If key fields affecting `equals/hashCode` mutate after insertion, retrieval fails.

B. Interface vs Abstract Class

Q6: Can interface have implementation?

Solution:

- Yes, with `default` and `static` methods.
- But it should still represent capability contract.

Q7: When abstract class is better?

Solution:

- When shared state, constructor logic, or protected utility methods are needed.

Q8: Can a class extend multiple abstract classes?

Solution:

- No. Java supports single class inheritance only.
- But class can implement multiple interfaces.

C. REST and HTTP

Q9: PUT vs PATCH

Solution:

- PUT replaces full resource representation.
- PATCH updates partial fields.
- Both can be idempotent when designed correctly.

Q10: 401 vs 403

Solution:

- 401: unauthenticated or invalid authentication.
- 403: authenticated but lacks permission.

Q11: Why use 409 Conflict?

Solution:

- For business-state conflicts (duplicate order, optimistic lock conflict).

Q12: Idempotency for POST payments

Solution:

- Accept `Idempotency-Key` .
- Persist request fingerprint + prior response.
- Return same result on retries.

D. Thread and Process

Q13: Difference between context switch of process vs thread

Solution:

- Process switch is heavier due to memory/address-space changes.
- Thread switch is lighter as threads share process memory.

Q14: Why can multithreading still be slower?

Solution:

- Lock contention, false sharing, context switching, IO bottlenecks.

Q15: How to fix race conditions?

Solution:

- Immutability, proper synchronization (`synchronized` , locks), atomics, message passing.

E. ArrayList and LinkedList

Q16: Why ArrayList often outperforms LinkedList even for inserts?

Solution:

- CPU cache locality and contiguous memory often dominate theoretical pointer-operation advantages.

Q17: Best list for frequent random read?

Solution:

- ArrayList.

Q18: Best list for frequent middle insertion by index?

Solution:

- Usually still evaluate carefully; LinkedList has $O(n)$ traversal to reach index.

F. Rapid Code Snippets (Interview-friendly)

Thread-safe counter with ConcurrentHashMap + LongAdder

```
ConcurrentHashMap<String, LongAdder> map = new ConcurrentHashMap<>();
map.computeIfAbsent("orders", k -> new LongAdder()).increment();
long count = map.get("orders").sum();
```

Strategy via Interface

```
interface TaxPolicy { BigDecimal apply(BigDecimal amount); }  
class IndiaTax implements TaxPolicy {  
    public BigDecimal apply(BigDecimal amount) { return amount.multiply(new  
BigDecimal("0.18")); }  
}
```

Idempotency key pseudo-flow

```
if (store.contains(key)) return store.get(key);  
Response r = process(request);  
store.put(key, r);  
return r;
```

Suggested Interview Practice Order

1. HashMap/ConcurrentHashMap internals and pitfalls.
 2. Interface vs abstract class design decisions.
 3. REST semantics + status code scenarios.
 4. Thread/process + synchronization basics.
 5. ArrayList/LinkedList with real workload reasoning.
-

Final Tip

For every answer, structure in 4 layers:

1. Definition
2. Internal mechanism
3. Production pitfall
4. Real-world example

This format consistently signals senior-level depth in Java interviews.